

Ch12 存储类别、链接和内存管理

对应《C Primer Plus（第6版）》第12章 · 本章目标：30分钟读懂 + 60分钟练完 · 电子信息方向后续课程与期末常见考点（static/extern、作用域存储期、malloc/free）

12.1 两个问题：变量「管到哪」和「活多久」

每个变量都有两个属性，初学时常被忽略但考试必考：

- **作用域**：在哪些代码里能用到它？——**块作用域**（函数或 {} 内声明，出了块就没了）vs **文件作用域**（函数外面声明，整个文件都能用，也叫全局变量）。
- **存储期**：它的内存什么时候分配、什么时候释放？——**自动存储期**（进入块分配、出块释放，默认的局部变量）vs **静态存储期**（程序启动分配、程序结束才释放）。

```
/* ch12_01_scope.c - 作用域演示 */
#include <stdio.h>

int count = 0;           /* 文件作用域 + 静态存储期（全局）*/

int main(void)
{
    int x = 1;           /* 块作用域 + 自动存储期 */
    {
        int x = 2;       /* 内层块自己的 x，遮蔽外层 */
        printf("内层 x = %d\n", x);
    }
    printf("外层 x = %d\n", x);
    printf("全局 count = %d\n", count);
    return 0;
}
```

12.2 static：让局部变量「记住」上一次的值

函数里普通的局部变量，每次调用都重新分配，但**不会自动初始化**——未赋初值的话，值是栈上残留的不确定值（本机实测同一函数两次调用分别得到 1 和 0）。加 static 后，它变成**静态存储期**：只初始化一次（默认置 0），函数退出不销毁，下次调用还留着上次的值。只有 static 变量与全局变量默认置 0。

```
/* ch12_02_static.c - static 计数器 */
#include <stdio.h>

int next_id(void)
```

```

{
    static int n = 0;    /* 只初始化一次! */
    n++;
    return n;
}

int main(void)
{
    printf("%d %d %d\n", next_id(), next_id(), next_id());
    return 0;
}

```

本机 gcc 16.1.0 实测输出: 3 2 1。C 标准不规定实参求值顺序（本机是从右向左），所以这种一行多次调用的写法不要用。若去掉 static 且保留初值 0，输出会是 1 1 1（每次调用重新初始化，与「记住上次值」无关）；若连初值都不给，值是栈上残留的不确定值并触发 -Wuninitialized 警告。

典型用途：计数器、只初始化一次的标志位、生成递增 ID（电子信息里给数据帧编号很常见）。

12.3 extern：使用别处定义的全局变量

extern 声明「这个变量在别处定义过，我这里只是引用」。同一个全局变量在多个文件/函数里共享时用它。单文件里，只要变量定义在前面，后面函数直接能用；跨文件时必须在使用方写 extern int count;。

```

/* ch12_03_extern.c - 跨函数共享全局（单文件演示）*/
#include <stdio.h>

int counter = 0;    /* 定义 */

void bump(void)    /* 函数里想用 counter: 其实同文件不用 extern */
{
    counter++;
}

int main(void)
{
    bump(); bump(); bump();
    printf("counter = %d\n", counter);
    return 0;
}

```

新手坑 1：全局变量不是越多越好。它让任何函数都能改，调试时不知道谁改的。课程阶段：少用全局，函数间传参优先（这也是 Ch10 讲 const 保护的原因）。

12.4 动态内存：malloc 和 free

前面学的数组长度都是**编译时就定死的**。运行时才知道要多大？用 malloc 在堆上按需申请：

```
/* ch12_04_malloc.c - 动态数组 */
#include <stdio.h>
#include <stdlib.h>      /* malloc/free 在这里 */

int main(void)
{
    int n, i;
    int *arr;

    printf("要几个数? ");
    scanf("%d", &n);
    arr = malloc(n * sizeof(int));      /* 申请 n 个 int; C 里无需 (int*) 强转 */
    if (arr == NULL) {                  /* 申请失败检查! */
        printf("内存不足\n");
        return 1;
    }
    for (i = 0; i < n; i++) arr[i] = i * i;
    for (i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);                          /* 用完必须还回去 */
    return 0;
}
```

新手坑 2：申请了不 free = 内存泄漏。循环里反复 malloc 不释放，内存越占越多，程序跑久了变慢甚至崩。养成「malloc 和 free 成对写」的习惯。

新手坑 3：free 之后再用。free 只是把内存还给系统，指针还指着那块地址——再去写就是未定义行为。free 后把指针置 NULL 是好习惯。

12.5 常见坑集中营

1. **static 初始化时机：**static 变量初始化只发生一次，且必须是编译期常量（不能用运行时值初始化）。
2. **忘了检查 malloc 返回值：**内存申请失败返回 NULL，直接用就崩。
3. **malloc 和 free 不配对：**申请 3 次只 free 1 次 = 泄漏；free 两次 = 双重释放崩溃。
4. **sizeof 用错：**malloc(n) 对 int 数组应该是 malloc(n * sizeof(int))，忘了乘 sizeof 会分配太小。

12.6 一分钟自测（答案见练习册）

判断对错：

- ① 函数内 static int n = 0; 每次调用函数时 n 都会被重置成 0。

② malloc 申请的内存用完必须 free，否则会泄漏。

③ 局部变量的作用域是它所在的整个文件。

12.7 练习指引

做完本章内容后，去练习册完成 Ch12 配套练习；同时打开《C Primer Plus（第6版）》第 12 章，完成**复习题 1、3、5**（题号对应主教材）。本章内容期末常考，static/extern 的理解建议用「把程序运行过程在脑子里演一遍」的方式自测。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译。原书题目请以正版教材为准。